Shopping Lists on Cloud

SDLE Project Presentation

João Sousa, up202106996
José Oliveira, up202108764
Alexandre Correia, up202007042

Context and Requirements

- This project implements a **local-first shopping** list application to ensure offline functionality allowing user to edit shopping lists without internet connection.
- Any user with access to the shopping list ID has permission to perform CRUD operations over the those shopping list items.
- **Data consistency** is ensured with the utilization of CRDTs to managed concurrent changes in the shopping lists.

Architecture

Client

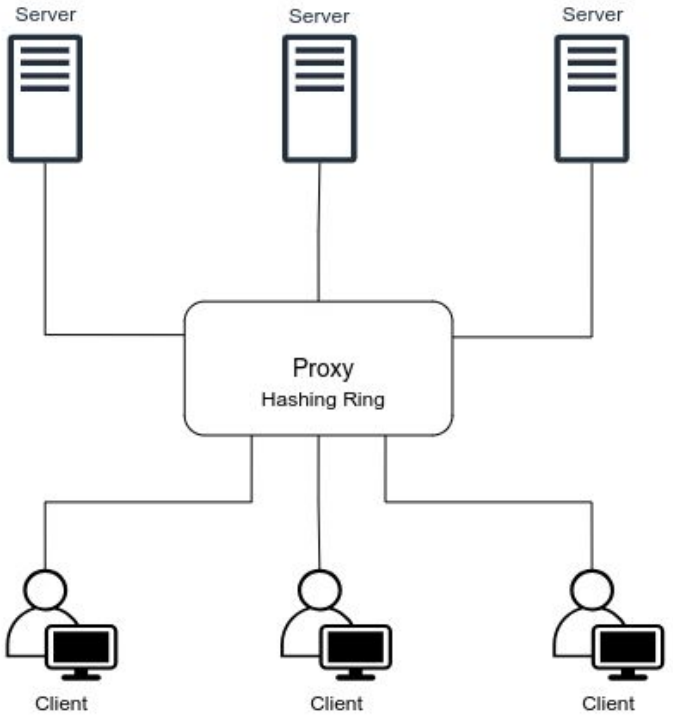
- request the creation and synchronization of shopping lists

Proxy

- provides the entry point for client to access the distributed system of servers
- contains the hashing ring used for data replication and request distribution

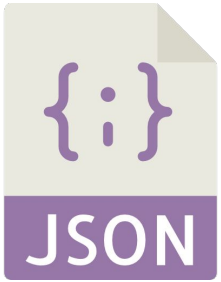
Server

- retrieve and update shopping list data



Technologies

- Python programming language
- ZeroMQ for messaging between client, server and proxy
- JSON Persistence
  - both local and cloud data stored in JSON files
  - automatic merging of local and remote states
- Multithreading
  - background thread ensures smooth synchronization



Shopping List CRDT

- **ShoppingList** class is a high-level manager for distributed shopping lists
  - create, update, delete lists and items
  - ensures consistency through merging remote and local data

- This class also provides functionality for loading data from JSON files containing information of each list.

- **AWORSet**: conflict-free set for managing items in lists
  - handles additions, deletions, and marking items as bought
  - maintains unique timestamps for conflict resolution

- **PNCounter**: counter for item quantities
  - supports independent increment and decrement operations
  - resolves conflicts by using maximum observed values

Client

- The local-first client manages shopping lists and synchronize them with a cloud backend combining **offline functionality** with cloud-based conflict resolution.

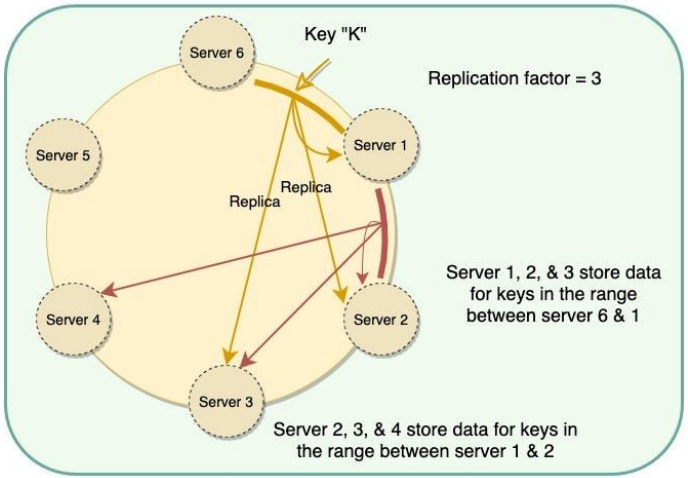
- Only the client can create shopping lists using UUID. Stores the changes made by the user when is offline and synchronizes when connection is regained.

Key Features

- Automatic Synchronization
  - background sync with the server every 15 seconds
  - handles conflicts using CRDTs (ShoppingList, AWORSet, and PNCounter)
- Terminal UI
  - simple console-based menu for list and item management.
  - tracks item quantities and purchase status.

Cloud – Proxy and Consistent Hashing

- The **Proxy** acts as middleware between clients and worker servers, efficiently routing and replicating data.



Key Features

- Consistent hashing is used with replicas for list-to-server mapping
- Load balancing across multiple worker servers
- Ensures high availability with data replication to neighboring servers

Cloud - Server

- This is the backend component responsible for managing shopping lists in the cloud and handling client/proxy requests ensuring data persistence and consistency across multiple nodes.

Key Features

Local Data Management

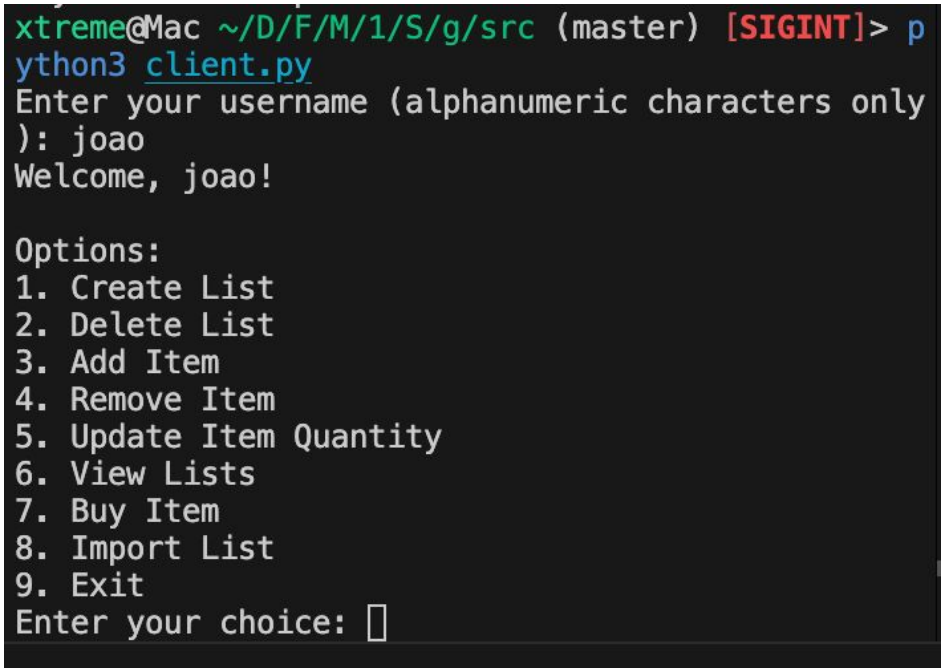
- JSON files for persistent storage
- Loads and merges data into a CRDT-based ShoppingList

Request Handling Actions:

- `syncLists`: merges data from clients into the server cloud shopping list
- `getListById`: retrieves a specific list by its unique ID

User Interface

- After the user logs in with an username, the UI allows the user to perform CRUD operations on shopping lists.
- Mark a list item as bought.
- Import a shopping list which was previously shared with the user.



Demo and Q&A

Thank you for your attention!